

CS222 Assignment 5: เรียนรู้ pthread part 2 (Synchronization)

Out: 7 เมษายน 69

400 คะแนน

Due: 26 เมษายน 69

Assignment นี้เป็นงานกลุ่มขนาดไม่เกิน 8 คน ที่อนุญาตให้ นศ จับกลุ่มกันเอง (ไม่จำเป็นต้องเป็นกลุ่มที่เคยกำหนดให้) หรือจะทำเดี่ยวก็ได้ มีวัตถุประสงค์ให้ศึกษา thread synchronization ด้วย pthread และ semaphore

นโยบายเกี่ยวกับการใช้ AI:

1. Snipped นโยบายเปลี่ยนไปตามรายวิชาและเป้าหมายการฝึกฝน นศ

นโยบายเกี่ยวกับกระบวนการเรียนรู้:

1. Snipped นโยบายเปลี่ยนไปตามรายวิชาและเป้าหมายการฝึกฝน นศ

การส่งงาน:

1. จงเขียนคำตอบใส่ในรายงานในรูปแบบไฟล์ pdf พร้อมทั้งเขียนชื่อและรหัส นศ ทุกคนในกลุ่มให้ชัดเจน และส่งงานในระบบ MS Team กำหนดให้ ทุก Assignment ต้องมี Assignment Header ที่ ระบุข้อมูลต่อไปนี้ที่ต้นรายงานอย่างชัดเจน ได้แก่

Assignment: Assignment ตามด้วยหมายเลข Assignment

Students: รายชื่อ นศ และ รหัส นศ ของผู้ร่วมงานทุกคน

หมายเหตุ: ต้องระบุรายชื่อที่ต้นรายงานเท่านั้นห้ามใส่ชื่อท้ายรายงานหรือที่อื่น

2. ในกรณีที่มีการคัดลอกงานหรือส่วนหนึ่งของงาน **ทั้งผู้คัดลอกและผู้ให้ลอก** จะได้รับ 0 คะแนน และจะถูกเตือน ถ้ายังมีการกระทำอีกจะมีการดำเนินการตามกฎหมายของมหาวิทยาลัย
3. ในกรณีส่งงาน late จะมีการตัดคะแนนวันละ 10 เปอร์เซนต์
4. เมื่อ upload ไฟล์ ครั้งแรกแล้ว นศ สามารถ upload ไฟล์เวอร์ชันใหม่ได้ โดยจะถือว่าเวลาในการ upload ครั้งล่าสุดเป็นเวลาส่ง (ถ้า upload ล่าสุดภายใน due date จะไม่ถูกหักคะแนน แต่ถ้าหลังจากนั้นจะถูกหักคะแนน)

สำหรับคำถามที่มีการเขียนโปรแกรม ให้ทำสิ่งต่อไปนี้

1. แนบ source code โดยให้เขียน comments อธิบาย code (ด้วยตนเอง) สั้นๆ ในรายงาน
2. capture ผลลัพธ์ของการรันโปรแกรมใส่ในรายงาน และอธิบายผลลัพธ์
3. นำ source code ของทุกข้อบรรจุใน tar file และ upload ขึ้น MS team
 - a. ดู [“How to use tar command in Linux”](#) หรือ this [youtube video](#)

Part 1: ศึกษา pthread mutex synchronization

1. จงศึกษา Slide 09 Synchronization เรื่อง Mutual exclusion และ Conditional Variable เพื่อเขียนโปรแกรมดังนี้ (15 คะแนน)
 - a. กำหนดให้มี global variable ชื่อ count มีค่าเริ่มต้นคือ count = 0
 - b. ให้ main thread สร้างเธรดใหม่ขึ้น 2 เธรด ให้ประมวลผลไปพร้อม ๆ กัน
 - c. เธรดที่ 1 เรียกฟังก์ชัน foo() ซึ่งจะในฟังก์ชันนี้ จะประมวลผลดังนี้
 - Lock(mutex)
 - เพิ่มค่า count ไปหนึ่งค่า และพิมพ์ค่า count ออกหน้าจอหลังจากการเพิ่มค่า
 - แจ้งบอกเธรดอื่นที่รอเงื่อนไข count > 0 อยู่ให้ตื่น
 - Unlock(mutex)
 - d. เธรดที่ 2 เรียกฟังก์ชัน bar() ซึ่ง ในฟังก์ชันนี้
 - Lock(mutex)
 - ถ้าตรวจได้ที่ค่า count <= 0 เธรดจะรอจนกว่าค่า count จะมากกว่า 0 แล้วจึงจะตื่นขึ้นมา (หลังจากที่เธรดอื่นแจ้งให้ตื่น) เพื่อลดค่า count ไปหนึ่งค่า และพิมพ์ค่า count ออกหน้าจอหลังจากการลดค่า
 - Unlock(mutex)

จงเขียนโปรแกรมเพื่อสร้าง 2 เธรดนี้ และใช้ conditional variables เพื่อจัดการ การประสานงานระหว่างเธรด ให้ capture source code และผลลัพธ์บนหน้าจอใส่ในรายงาน

2. จากโปรแกรมใน **ข้อ-4 ข้อ 1** จงสร้างเธรดขึ้น 8 เธรดให้ประมวลผลพร้อมกัน กำหนดให้ 4 เธรดเรียกฟังก์ชัน foo() และอีก 4 เธรดเรียกฟังก์ชัน bar() ให้ capture source code และผลลัพธ์ใส่ในรายงาน และตอบว่าท้ายที่สุดแล้วค่าของ count คืออะไร (10 คะแนน)

3. จงตอบว่าเพราะเหตุใดการเขียนโปรแกรมแบบ conditional variables ในตัวอย่างใน Slide 09 Synchronization บรรทัดที่ 16-18 จึงต้องใช้ while loop (10 คะแนน)

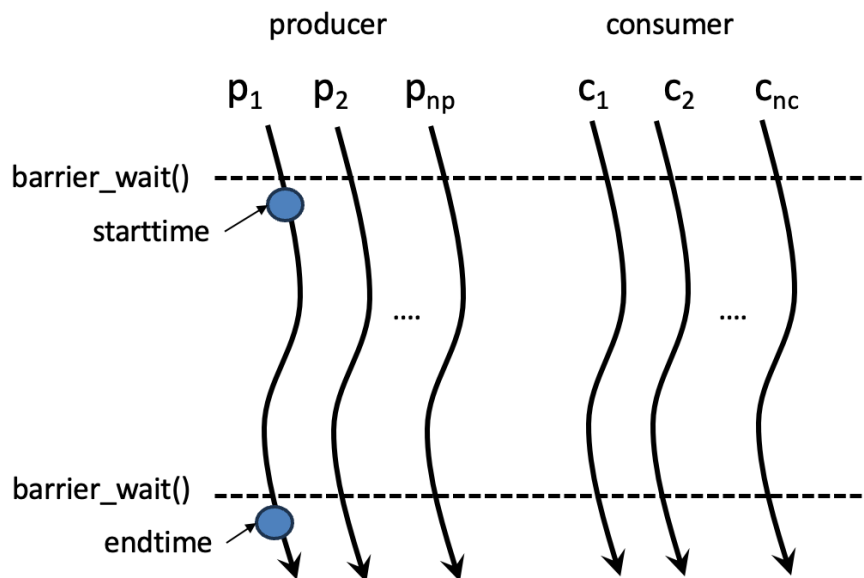
```
16 while(global_var <= 0){
17     pthread_cond_wait(&cv, &mutex_var);
18 }
```

4. จงค้นคว้าจาก [Textbooks “Operating System Concepts”](#) และ [“Operating Systems: Three Easy Pieces”](#) เพื่อตอบว่า ปัญหา Classic Synchronization Problems ต่อไปนี้คืออะไร และต้องใช้เครื่องมือใดในการแก้ปัญหา (15 คะแนน)
- Bounded Buffer Problem
 - Reader Writer Problem
 - Dining Philosopher Problem
5. จงเขียนและรันโปรแกรมมัลติเทรตโดยมีข้อกำหนดต่อไปนี้ (20 คะแนน)
- ให้สร้างโปรแกรมชื่อ asg6-5 ให้รับค่า arguments ทาง Command Line ดังนี้
\$./asg6-5 n
โดยที่ n คือจำนวนเทรตในโปรแกรม
 - ให้ Main thread สร้างเทรต n เทรต โดยที่แต่ละเทรตเรียกใช้ starting routine ชื่อ function foo ซึ่งเป็นฟังก์ชันที่รับค่า interger เป็นค่า Rank number ตั้งแต่ 1 ถึง n
 - หลังจากสร้างเทรต foo ทั้งหมดแล้ว ให้ Main thread รอให้เทรตทั้งหมดจบการทำงานโดยใช้คำสั่ง pthread_join()
 - เมื่อเริ่มการทำงานให้เทรต foo แต่ละเทรต sleep เป็นเวลาเท่ากับค่า Rank number ที่ผ่านให้เป็นค่าพารามิเตอร์
 - หลัง sleep เสร็จ ให้ foo พิมพ์ค่า
 - เวลาปัจจุบัน: ให้ นศ ค้นหาวิธีการพิมพ์วันเวลาปัจจุบันในภาษา C ออกสู่หน้าจอด้วยตนเอง Hint: gettimeofday
 - thread id และ
 - rank numberออกสู่หน้าจอ
 - ให้ foo เรียกใช้คำสั่ง pthread_barrier_wait() ให้ศึกษาการใช้งาน interface นี้ด้วยตนเอง
 - หลังจากนั้น ให้ foo พิมพ์ค่า

- เวลาปัจจุบัน: ให้ นศ ค้นหาวิธีการพิมพ์วันเวลาปัจจุบันในภาษา C ออกสู่หน้าจอด้วยตนเอง
- thread id และ
- rank number

ออกสู่หน้าจอ

- ห. จงค้นหาวิธีคำนวณ ความต่างของเวลาก่อนและหลังเรียก pthread_barrier_wait ของแต่ละเธรด (rank 1 ถึง 4)
 - ิ. ให้แต่ละเธรดพิมพ์ค่าความแตกต่างของเวลาดังกล่าวแล้ววิเคราะห์ว่าเป็นเพราะอะไรเวลาจึงเหมือนหรือต่างกัน
6. จากตัวอย่าง producer consumer ของ bounded buffer (แบบ circular queue) โดยใช้เครื่องมือ mutex lock/unlock และ conditional variables ดังตัวอย่างโปรแกรมใน Slide 09 จงเขียนโปรแกรมตามกำหนดต่อไปนี้ (60 คะแนน)
- ให้สร้างโปรแกรมชื่อ asg6-6 ให้รับค่า arguments ทาง Command Line ดังนี้
\$./asg6-6 np nc nx
 โดยที่ **np** คือจำนวนเธรด producer **nc** คือจำนวนเธรด consumer และ **nx** คือจำนวนรอบการเพิ่มและลบข้อมูลออกจาก bounded buffer โปรแกรมนี้ภาพรวมการทำงานดังภาพที่ 1



ภาพที่ 1

- b. Main thread สร้าง producer thread จำนวน np เทรด แต่ละเทรดจะมีค่า rank number ผ่านให้เป็นพารามิเตอร์ ตั้งแต่ 1 ถึง np ตามลำดับ producer เทรดต้องรันพร้อมกัน
- c. Main thread สร้าง consumer thread จำนวน nc เทรด แต่ละเทรดจะมีค่า rank number ผ่านให้เป็นพารามิเตอร์ ตั้งแต่ 1 และ nc ตามลำดับ consumer เทรดต้องรันพร้อมกัน
- d. หลังจากที่ Main thread สร้าง producer และ consumer เทรดทั้งหมดแล้ว ให้ Main thread ใช้คำสั่ง pthread_join() เพื่อรอให้เทรดทั้งหมดจบการทำงาน
- e. กำหนดให้ Producer และ Consumer thread ทั้งหมด ใช้ pthread_barrier_wait() synchronization เพื่อให้ทุกเทรดประมวลผลคำสั่ง ถัดจากคำสั่ง pthread_barrier_wait() พร้อมกัน
 - ให้นักศึกษาการใช้งาน pthread_barrier_wait() ด้วยตนเองจาก Slide 9 และจาก AI หรือ Internet
- f. หลังจากผ่าน barrier synchronization แล้ว ให้ producer thread ที่มีค่า rank id เท่ากับ 1 ใช้ฟังก์ชัน เช่น gettimeofday() เพื่อหาค่าเวลาและบันทึกค่าเวลาสมมุติว่าชื่อ starttime ทันที
 - ให้นักศึกษาการใช้งาน gettimeofday() ด้วยตนเองจาก จาก AI หรือ Internet
- g. ให้ producer thread แต่ละเทรดวน loop เพิ่มค่าเข้าสู่คิวด้วยฟังก์ชัน en_Q(data) เป็นจำนวน nx ครั้ง และกำหนดให้ค่า data ของแต่ละเทรดเท่ากับค่า rank number ของเทรดนั้น นอกจากนั้น ในการวน loop เพิ่มค่าในคิวดังกล่าว แต่ละ producer thread จะต้อง sleep เป็นเวลาเท่ากับ 1 วินาที ก่อนที่จะเพิ่มค่าเข้าสู่คิว เมื่อเพิ่มค่าในคิวแล้วให้พิมพ์ “produce” ตามด้วยค่า rank id ออกสู่หน้าจอ
- h. หลังจากผ่าน barrier synchronization แล้ว กำหนดให้ consumer thread แต่ละเทรดวน loop เพื่ออ่านค่าออกจากคิว จำนวน $ny = [(np \times nx)/nc]$ ครั้ง (ยกตัวอย่างเช่น ถ้า $np = 8, nx = 10, nc = 2$ จะได้ $ny = 40$ เป็นต้น) เมื่ออ่านค่าจากคิวแล้วให้พิมพ์ “consume” ตามด้วยค่า rank id ของ consumer thread และค่าข้อมูลที่อ่านจากคิวออกสู่หน้าจอ
- i. ก่อนที่ Producer และ consumer thread จะจบการทำงาน กำหนดให้ producer และ consumer thread ทั้งหมด เรียก pthread_barrier_wait() ครั้งที่สอง เพื่อรอให้ทุกเทรดจบการทำงานทั้งหมดก่อนจึงจะทำงานต่อ

- j. หลังจากผ่าน barrier synchronization แล้ว ให้ producer thread ที่มีค่า rank id = 1 ใช้ฟังก์ชัน เช่น gettimeofday() เพื่อ หาค่าเวลาและบันทึกค่าเวลาสมมุติว่าชื่อ endtime ทันที
- k. ให้เทรต producer rank 1 นำค่าเวลาทั้งสองมาหักลบกันเพื่อคำนวณหาค่าระยะเวลาที่โปรแกรมใช้ตั้งแต่เริ่มมีการใส่ค่าเข้าสู่คิว จนกระทั่งถึงเวลาที่เทรตสุดท้ายอ่านค่าออกจากคิว และพิมพ์ค่าเวลาดังกล่าวออกสู่หน้าจอ
- l. ให้รันโปรแกรม โดยมีพารามิเตอร์ดังตารางต่อไปนี้ และแนบผลลัพธ์ในรายงาน **จงอธิบายอย่างสั้นว่าเพราะเหตุใดผลลัพธ์จึงเป็นเช่นนั้น**

run	<i>np</i>	<i>nc</i>	<i>nx</i>
1	10	10	10
2	10	1	10
3	1	10	10
4	8	2	10
5	2	8	10

- m. ให้เขียนตารางในรายงานแล้วเพิ่มอีกหนึ่ง column เพื่อรายงานระยะเวลาที่เทรต producer rank = 1 ประมวลผล ระบุหน่วยของเวลาให้ชัดเจน
7. ศึกษา Slide 09 Synchronization เรื่อง **Conditional Variable** จงเขียนโปรแกรมเพื่อจำลองการทำงานของ pthread_barrier_wait() โดยใช้ conditional variable ดังนี้ (40 คะแนน)
- ให้สร้างโปรแกรมชื่อ asg6-7 ให้รับค่า arguments ทาง Command Line ดังนี้
\$./asg6-7 n
 โดยที่ *n* คือจำนวนเทรต
 - กำหนดให้มี global variable ชื่อ count มีค่าเริ่มต้นคือ **count = 0**
 - ให้เทรต main สร้างเทรต *n* เทรต ให้ประมวลผลพร้อมกัน และใช้ pthread_join() เพื่อรอทุกเทรตให้จบการประมวลผล
 - ทุกเทรตเรียกฟังก์ชัน foo() ซึ่งจะในฟังก์ชัน จะประมวลผลดังนี้
 - Lock(mutex)
 - เพิ่มค่า count ไปหนึ่งค่า และพิมพ์ค่า count ออกหน้าจอหลังจากการเพิ่มค่า
 - ถ้า ทราบใดที่ค่า count < *n* เทรตจะรอจนกว่าค่า count จะเท่ากับ *n* แล้วจึงจะตื่นขึ้นมา (หลังจากที่เทรตอื่นแจ้งให้ตื่น) เพื่อพิมพ์ค่า count ออกหน้าจอ แล้วจึง unlock(mutex) และพิมพ์คำว่า “Done” บนหน้าจอและจบการประมวลผลของเทรตนั้น

- แต่ถ้าค่า `count == n` ก็ให้ แจ้งบอกเทรดอื่นที่รอเงื่อนไข `count == n` อยู่ให้
ตื่น

แล้วหลังจากนั้นเทรดจะ `unlock(mutex)` และพิมพ์คำว่า “Done” บนหน้าจอและ
จบการประมวลผล

- ให้รันโปรแกรม โดยมีพารามิเตอร์ $n = 2$ $n = 4$ และ $n = 8$ และ capture ผลลัพธ์
บนหน้าจอใส่ในรายงาน พร้อมทั้งอธิบายสั้นๆว่าโปรแกรมนั้นจำลอง `pthread_barrier_wait()`
อย่างไร

Part 2: ศึกษา semaphore synchronization

- จงศึกษา Semaphore เพิ่มเติมโดยอ่าน [บทที่ 31 ของ OSTEP textbook](#) และตอบคำถามต่อไปนี้
(20 คะแนน)

- จงอธิบายว่า ตัวแปรแบบ `sem_t` คืออะไร และ interface `sem_init()` `sem_wait()` และ
`sem_post()` คืออะไร มีกลไกในการทำงานอย่างไร
- ให้ใช้ `pthread` สร้างเทรด 2 เเทรด ที่มี starting routine คือ function `foo()` และ `bar()` ให้
เริ่มประมวลผลพร้อมกัน จงใช้ semaphore เพื่อทำให้ `foo()` พิมพ์ข้อความ “this is foo.”
ก่อนที่ `bar()` จะพิมพ์ข้อความ “this is bar.” เสมอ

- จงเขียนโปรแกรมเพื่อจำลองการเข้าทานอาหารใน ร้านอาหาร ที่มีที่นั่งทานอาหารเพียง n ที่นั่งใน
ขณะที่มีลูกค้ามาทานอาหาร m คน โดยให้สร้างเทรดหนึ่งเทรดแทนลูกค้าแต่ละคน ลูกค้าแต่ละคน
จะใช้เวลาทานอาหาร x วินาที จงตอบคำถามต่อไปนี้ (50 คะแนน)

- จงเขียนโปรแกรม multi-thread ที่ใช้ semaphore จำลองการเข้าทานอาหารในร้านอาหาร
นี้ สมมุติว่าโปรแกรมชื่อ `asg6-9` ให้รันโปรแกรมบน Command Line โดยใช้คำสั่ง

\$./asg6-9 ns nc et

โดยที่ **ns** คือจำนวนที่นั่งในร้าน (number of seats) **nc** คือจำนวนเทรดที่จำลองเป็น
ลูกค้า (number of customers) และ **et** เวลาที่ลูกค้าแต่ละคนนั่งทานอาหาร (eating
time) หน่วยเป็นวินาที

- กำหนดให้เทรด main สร้าง เเทรด customer จำนวน **nc** เเทรด โดยแต่ละเทรดมี rank id
เท่ากับ 1 ถึง **nc** ตามลำดับ และให้เทรด main เรียก `pthread_join()` เพื่อรอให้ทุกเทรดจบ
การประมวลผล
- ก่อนเทรดเรียก `sem_wait()` ให้พิมพ์ “--> rank id %d is waiting for a seat\n” และ
หลังจากคำสั่ง `sem_wait()` ให้พิมพ์ “--> rank id %d got a seat\n”
- ใน Critical Section ให้เทรด sleep เป็นเวลา **et** วินาทีเพื่อจำลองการทานอาหาร

- e. ก่อนเรียก `sem_post()` ให้พิมพ์ “<-- rank id %d is leaving\n”
- f. เทรด customer จบการประมวลผล หลังจากคำสั่ง `sem_post()`
- g. ให้รันโปรแกรม โดยมีพารามิเตอร์ดังตารางต่อไปนี้ และแนบผลลัพธ์ในรายงาน **จงอธิบายอย่างสั้นว่าเพราะเหตุใดผลลัพธ์จึงเป็นเช่นนั้น**

run	<i>ns</i>	<i>nc</i>	<i>et</i>
1	5	10	1
2	10	5	1
3	1	10	1
4	10	10	1
5	0	5	1

10. สมมุติว่า ค่าเริ่มต้นของตัวแปร Semaphore คือ **1** (จำ) โดยที่ จงเขียนโปรแกรมให้เทรด main สร้างเทรด `foo()` และ `bar()` และให้ main รอให้เทรดทั้งสองจบการประมวลผล จงทดสอบพฤติกรรมของ semaphore synchronization ดังต่อไปนี้ (15 คะแนน)
 - a. เทรด main รับ argument 1 ค่า ทาง Command Line คือ ***n*** เป็นเลขจำนวนเต็มบวก
 - b. จงเขียนโปรแกรมให้เทรด `foo()` วง loop รันคำสั่ง `sem_post()` จำนวน ***n*** ครั้ง แล้วให้เทรด `foo()` sleep เป็นเวลา 10 วินาที แล้วจบการประมวลผล
 - c. ในขณะเดียวกันให้เทรด `bar()` เริ่มโย sleep เป็นเวลา 3 วินาที แล้ววน loop เพื่อรันคำสั่ง `sem_wait()` forever
 - d. จงตอบว่าเทรด `bar()` จะต้องเรียก `sem_wait()` กี่ครั้ง `sem_wait()` จึงจะ block ส่งผลให้เทรด `bar()` รอตลอดไป จนกระทั่งผู้ใช้ กด Ctrl C เพื่อ terminate โปรแกรม
11. สมมุติว่า โปรแกรมมีตัวแปร mutex หนึ่งตัวแปร จงเขียนโปรแกรมให้เทรด main สร้างเทรด `foo()` และ `bar()` และให้ main รอให้เทรดทั้งสองจบการประมวลผล จงทดสอบพฤติกรรมของ `pthread_mutex_lock` และ `unlock` ดังต่อไปนี้ (15 คะแนน)
 - a. เทรด main รับ argument 1 ค่า ทาง Command Line คือ ***n*** เป็นเลขจำนวนเต็มบวก
 - b. จงเขียนโปรแกรมให้เทรด `foo()` วง loop รันคำสั่ง `pthread_mutex_lock()` บน mutex variable จำนวน ***n*** ครั้ง แล้วให้เทรด `foo()` sleep เป็นเวลา 10 วินาที แล้วจบการประมวลผล
 - c. ในเดียวกันให้เทรด `bar()` เริ่มโดย sleep เป็นเวลา 3 วินาที แล้ววน loop เพื่อรันคำสั่ง `pthread_mutex_lock()` บน mutex variable forever

- d. จงตอบว่าเทรต `bar()` จะต้องเรียก `pthread_mutex_lock()` กี่ครั้ง จึงจะ block หมดไปจนกระทั่งผู้ใช้ กด Ctrl C เพื่อ terminate โปรแกรม
12. จงอธิบายว่าเพราะเหตุใดพฤติกรรมของ `pthread_mutex_unlock()` ในข้อ 11 จึงแตกต่างจากพฤติกรรมของ `sem_post()` ในข้อ 10 (10 คะแนน)
13. จาก [บทที่ 31.4 ของหนังสือ OSTEP](#) จงตอบว่า การกำหนดค่าเริ่มต้นให้ตัวแปร Semaphore เท่ากับ 1 และการกำหนดค่าเริ่มต้นให้ตัวแปรนั้นเป็น 0 ส่งผลให้พฤติกรรมการทำงานของโปรแกรมต่างกันอย่างไร (Hint: พิจารณาโดยทำความเข้าใจตัวอย่างง่ายๆว่าพฤติกรรมที่รอตต่างกันอย่างไร) (10 คะแนน)
14. จงศึกษาการใช้ semaphore เพื่อแก้ปัญหา Bounded Buffer [ในบทที่ 31.4 ของหนังสือ OSTEP](#) และทำสิ่งต่อไปนี้ (40 คะแนน)

- a. จงเขียนโปรแกรมตาม Code ใน Figure 3.9 และ 31.11 (ภาพที่ 2) จงอธิบายว่าโปรแกรมมีข้อผิดพลาดอย่างไร

```
1 void *producer(void *arg) {
2     int i;
3     for (i = 0; i < loops; i++) {
4         sem_wait(&mutex);           // Line P0 (NEW LINE)
5         sem_wait(&empty);           // Line P1
6         put(i);                     // Line P2
7         sem_post(&full);            // Line P3
8         sem_post(&mutex);           // Line P4 (NEW LINE)
9     }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&mutex);           // Line C0 (NEW LINE)
16         sem_wait(&full);            // Line C1
17         int tmp = get();             // Line C2
18         sem_post(&empty);           // Line C3
19         sem_post(&mutex);           // Line C4 (NEW LINE)
20         printf("%d\n", tmp);
21     }
22 }
```

Figure 31.11: Adding Mutual Exclusion (Incorrectly)

ภาพที่ 2

- b. จงศึกษา “Avoiding Deadlock” ในหน้าที่ 10 ของบทที่ 31.4 แล้วรันเทรต producer และ consumer อย่างละ 1 เทรต แล้วเพิ่มคำสั่ง `sleep` แทรกลงใน code เพื่อจงใจทำให้โปรแกรมเกิดสถานการณ์ deadlock ดังที่อธิบายใน Textbook
- c. จงเขียนโปรแกรมตาม Code ใน Figure 31.9 และ 31.12 ใน Textbook และรันโปรแกรมเพื่อแสดงว่าโปรแกรม Bounded buffer ที่ไม่เกิดปัญหา deadlock

15. จงศึกษาการใช้ semaphore เพื่อแก้ปัญหา Dining Philosophers [ในบทที่ 31.6 ของหนังสือ OSTEP](#) (หน้า 13) และทำสิ่งต่อไปนี้ (40 คะแนน)

- a. จงเขียนโปรแกรมสมมติว่าชื่อ asg6-15 ให้รับคำสั่งทาง Command Line คือ

```
$ ./asg6-7 n
```

โดยที่ n คือจำนวน philosophers และ k คือจำนวนรอบของการกิน

- b. ให้สร้างเทรตจำนวน n เทรต โดยที่ $n > 1$ และผ่านค่า rank id ตั้งแต่ 1 ถึง n เป็นค่าพารามิเตอร์ของแต่ละเทรต

- c. กำหนดให้แต่ละเทรต จำลองกิจวัตรของ philosopher แต่ละคน คือ จะวน loop จำนวน k ครั้งเพื่อ

```
think();
```

```
พิมพ์ "--> Rank %d WAITs to get fork\n", rank
```

```
get_fork();
```

```
พิมพ์ "---->Rank %d GOT fork\n", rank
```

```
eat();
```

```
พิมพ์ "<-- Rank %d PUT to get fork\n", rank
```

```
put_fork();
```

และจำลอง think() ด้วยการ sleep 1 วินาที และ eat() ด้วยการ sleep 2 วินาที

- d. จงเขียน Code ตาม Figure 3.15 และ 31.16 และรันโปรแกรม โดยป้อนค่าพารามิเตอร์ต่อไปนี้ และแนบผลลัพธ์ในรายงาน

- e. จงอธิบายอย่างสั้นว่าเพราะเหตุใด Code ใน Figure 31.16 จึงทำให้ไม่เกิด deadlock และวิธีการแก้ปัญหาใน Figure 31.16 ตรงกับหลักการเขียนโปรแกรมแบบ Deadlock Prevention หรือไม่

16. จงศึกษาการ Implement semaphore โดยใช้ pthread_mutex_lock/unlock และ conditional variable [ในบทที่ 31.8 ของหนังสือ OSTEP](#) (หน้า 16) (30 คะแนน)

- a. จงเขียนโปรแกรมเพื่อจำลอง semaphore ตาม code ใน Figure 31.17

- b. จงเขียนโปรแกรมเพื่อสร้างเทรต สองเทรต และใช้กลไก semaphore ที่จำลองขึ้น เพื่อแสดงว่าการประสานงานทำได้ถูกต้อง อธิบายอย่างสั้นว่าทำไมผลการรันถึงถูกต้อง